

Examen Final

Paradigmas de Programación (plan 2023)
 Departamento de Computación, FCEyN, UBA
 1 de agosto de 2025

1	2	3	4	Nota
---	---	---	---	------

Incluir nombre, fecha y firma en todas las hojas.

Ejercicio 1 (Razonamiento ecuacional)

Consideramos el tipo de datos de los árboles binarios:

```
data AB a = Nil | Bin (AB a) a (AB a)
```

y las siguientes funciones:

```
{P0} post Nil          = []
{P1} post (Bin i x d)  = post d ++ post i ++ [x]
{M0} map f []          = []
{M1} map f (x : xs)    = f x : map f xs
{A0} mapA f Nil        = []
{A1} mapA f (Bin i x d) = [f x] ++ mapA f i ++ mapA f d
{R0} rev ac []         = ac
{R1} rev ac (x : xs)   = rev (x : ac) xs
```

Demostrar que para toda $f :: a \rightarrow b$ vale

$$\text{map } f \cdot \text{post} = \text{rev } [] \cdot \text{mapA } f.$$
Ejercicio 2 (Programación funcional / resolución)

Suponemos que contamos con los siguientes tipos de datos y funciones en Haskell para modelar la resolución SLD:

- `CláusulaDeDefinición` — tipo de las cláusulas de definición.
- `CláusulaObjetivo` — tipo de las cláusulas objetivo.
- `esVacía :: CláusulaObjetivo -> Bool` — función que indica si una cláusula es $\square$.
- `resolvente :: CláusulaObjetivo -> CláusulaDeDefinición -> CláusulaObjetivo` — función que calcula la resolvente entre dos cláusulas.

Definir una función

```
existeRefutaciónSLD :: [CláusulaDeDefinición] -> CláusulaObjetivo -> Bool
```

que indique si existe una refutación SLD. En caso de que la refutación exista, la función debe devolver `True`. En caso de que la refutación no exista, la función debe devolver `False`, o posiblemente indefinirse (por no terminación).

Ejercicio 3 (Deducción natural)

Extendemos las fórmulas de la lógica de primer orden con un nuevo cuantificador $\odot x.\tau$ con las siguientes reglas de introducción y eliminación:

$$\frac{\Gamma, \neg\tau\{x := t\} \vdash \perp}{\Gamma \vdash \odot x.\tau} \odot_i \qquad \frac{\Gamma \vdash \odot x.\tau \quad \Gamma \vdash \neg\tau \quad x \notin \text{fv}(\Gamma)}{\Gamma \vdash \perp} \odot_e$$

Demostrar que $\vdash (\odot x.P(f(x))) \Rightarrow \odot x.P(x)$.

Ayuda: el significado intuitivo del nuevo cuantificador es equivalente al del cuantificador existencial usual.

Ejercicio 4 (Programación lógica)

Representamos en Prolog las fórmulas de la lógica proposicional con los símbolos de función binarios **and** (conjunción), **or** (disyunción), **imp** (implicación), el símbolo de función unario **neg** (negación), y el símbolo de función unario **prop** de tal modo que **prop(n)** representa la n -ésima variable proposicional. Por ejemplo, la fórmula $P_1 \vee \neg(P_2 \wedge \neg P_2)$ se representa como:

```
or(prop(1), neg(and(prop(2), neg(prop(2)))))
```

Se puede asumir que en una fórmula **prop(n)** el valor de n siempre es positivo. Definir un predicado **esTautologia(+Form)** que, dada una fórmula **Form**, sea verdadero si todas las valuaciones la satisfacen.